BANCO DE DADOS · DOCUMENTAÇÃO TÉCNICA

Aeroporto-BD

Explicação do projeto e do código: arquitetura, banco de dados e as partes mais importantes da implementação em Java.

Sistema de gestão do Aeroporto Internacional do Recife

Mariana Maliu · Arthur Coelho · Miguel Tojal · Raul Maia · Arthur Queiroz · Vitor Gadelha

github.com/mtojald/Aeroporto-BD

VISÃO GERAL

O que é o Aeroporto-BD

O Aeroporto-BD é um sistema de gestão para um aeroporto fictício (inspirado no Aeroporto Internacional do Recife): voos, aeronaves, companhias aéreas, tripulação, passageiros, bilhetes, bagagens e manutenções. O projeto nasceu como trabalho da disciplina de Banco de Dados e foi construído em três etapas que se encaixam: primeiro o **modelo** (entidade-relacionamento), depois o **banco** (20 tabelas em MySQL) e por fim a **aplicação** (interface web em Java que lê e escreve nesse banco).

Este documento cobre principalmente a terceira etapa — o código da aplicação — explicando como ele está organizado e detalhando os trechos mais importantes: a conexão com o banco, o roteamento HTTP, a camada de acesso a dados (DAO), o tratamento de erros do MySQL, as transações e os cálculos do dashboard.

Pilha de tecnologia

- **Java 17** puro, sem framework web (Spring, etc.) — o próprio servidor HTTP do JDK (`com.sun.net.httpserver.HttpServer`) atende as requisições.
- **JDBC** com o driver `mysql-connector-j` — o SQL é escrito à mão no código, sem ORM (nada de Hibernate/JPA).
- **MySQL 8** como banco relacional, com 20 tabelas, chaves estrangeiras, `CHECK`, `UNIQUE` e índices auxiliares.
- **HTML, CSS e JavaScript puro** no front-end, sem React/Vue — a página conversa com o backend por `fetch` chamando uma API REST que devolve JSON, e usa **Chart.js** para os gráficos do dashboard.

Estrutura de pastas

Pasta	Conteúdo
database/	migrations.sql (cria o banco), queries.sql (popula) e consultas.sql (as 10 consultas)
src/aeroporto/db/	Acesso bruto ao banco: conexão (Database), tradução de erros (ErroSql), leitura de resultados (Linhas)
src/aeroporto/model/	Records que representam uma linha de tabela (Voo, Passageiro)
src/aeroporto/dao/	Um DAO por funcionalidade: o SQL de cada operação vive aqui (VooDAO, PassageiroDAO, DashboardDAO, ConsultaDAO, OpcoesDAO)
src/aeroporto/web/	Camada HTTP: handlers REST, parsing/validação de formulário, serialização JSON
web/	Front-end estático: index.html, app.js, style.css, dashboard.css e o Chart.js vendorizado

ARQUITETURA

Como as camadas se conversam

A aplicação segue uma arquitetura simples em camadas, sem abstrações além do necessário. Cada camada tem uma responsabilidade única e só conhece a camada imediatamente abaixo dela:

Camada	Responsabilidade	Classes/arquivos
Navegador	HTML + JS puro; monta a tela, valida o básico e chama a API	web/index.html, app.js
Handler	Traduz HTTP ↔ Java: lê a URL/formulário, decide o status HTTP, devolve JSON	web/*Handler.java
DAO	Guarda o SQL de cada operação e conversa com o JDBC	dao/*DAO.java
JDBC	Driver do MySQL: executa o SQL de fato	mysql-connector-j
MySQL	20 tabelas com chaves, constraints e índices	database/migrations.sql

Uma requisição típica — por exemplo, excluir um passageiro — percorre essas camadas assim: o navegador chama `DELETE /api/passageiros/123.456.789-00`; o `PassageiroHandler` (que estende `CrudHandler`) identifica o método e o id na URL e chama `PassageiroDAO.excluir(cpf)`; o DAO roda o `DELETE` via JDBC; se o MySQL recusar porque o passageiro tem bilhetes, o erro sobe como exceção e vira uma mensagem legível na tela — sem o processo inteiro quebrar.

> **Sem ORM, por quê?**
>
> O projeto evita um ORM de propósito: o objetivo é mostrar o SQL explicitamente, como parte da avaliação da disciplina de Banco de Dados. Isso tem um custo (mais código repetitivo nos DAOs), mas deixa claro exatamente qual consulta roda em cada operação — o que é o ponto central do projeto.

Pacote aeroporto.db — a base de tudo

Três classes pequenas sustentam toda a comunicação com o banco:

- **Database** — ponto único de conexão. Lê usuário/senha de `config.properties` (com fallback para o arquivo de exemplo) e abre uma `Connection` nova a cada chamada.
- **ErroSql** — traduz códigos de erro do MySQL (1062, 1451, 1452, 3819...) em mensagens em português e no status HTTP correto.
- **Linhas** — converte um `ResultSet` inteiro em `List<Map<String,Object>>`, pronto para virar JSON.

```
public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection(
            config("db.url", "jdbc:mysql://localhost:3306/aeroporto"),
            config("db.user", "root"),
            config("db.password", ""));
}
```

Database.java — toda chamada ao banco passa por aqui; não há pool de conexões (escopo didático).

BANCO DE DADOS

O esquema por trás do código

O banco tem 20 tabelas (`database/migrations.sql`) desenhadas para cobrir, de propósito, os principais conceitos de modelagem relacional. Isso molda bastante o código Java: cada particularidade do modelo aparece como uma regra específica em algum DAO.

Conceito de modelagem	Onde aparece	Reflexo no código Java
Entidade fraca	Portao (de Terminal), Assento e Manutenção (de Aeronave), Bagagem (de Bilhete)	Chave composta: ex. Voo.ler() recebe “portao-terminal” do front e separa os dois valores
Especialização	Funcionario → Piloto, Comissario, Mecanico (PK = FK)	Consultas fazem JOIN entre a subclasse e Funcionario para montar o nome completo
N:N	Escala (Funcionario × Voo, com funcao_no_voo)	ConsultaDAO #6 usa GROUP_CONCAT para listar a tripulação de um voo
Auto-relacionamento	Voo.fk_Voo_conexao aponta para outro Voo	VooDAO.excluir() zera essa coluna nos voos dependentes antes de apagar
Atributo multivalorado	Passageiro_Telefone, Funcionario_Telefone, Comissario_Idioma	PassageiroDAO grava a lista de telefones numa transação à parte, com executeBatch
Integridade	FOREIGN KEY, UNIQUE, CHECK, ON DELETE CASCADE	ErroSql traduz a violação (ex. 1451) numa mensagem amigável no lugar de um stack trace

Por que isso importa para o código

Boa parte da complexidade dos DAOs não é “lógica de negócio” arbitrária — é a aplicação respeitando o modelo relacional. Por exemplo, excluir um Voo precisa de uma transação com duas instruções porque o auto-relacionamento (conexão) deixaria uma chave estrangeira “pendurada” se o voo referenciado sumisse primeiro.

CAMADA WEB

CrudHandler: o roteador REST genérico

A peça mais importante da camada web é `CrudHandler`, uma classe abstrata que implementa o roteamento REST uma única vez e é reaproveitada por todos os recursos (`VooHandler`, `PassageiroHandler`, `DashboardHandler` etc.). Ela decide o verbo HTTP, extrai o id da URL e converte qualquer erro em uma resposta JSON com o status certo — cada subclasse só precisa implementar os métodos que realmente suporta.

```
public final void handle(HttpExchange ex) throws IOException {
    try {
        String id = extrairId(ex.getRequestURI().getRawPath());
        Map<String,String> campos;
        switch (ex.getRequestMethod()) {
            case "GET" -> {
                semId(id);
                Http.enviarJson(ex, 200, listar());
            }
            case "POST" -> {
                semId(id);
                campos = Http.lerFormulario(ex);
                Http.enviarJson(ex, 201, Map.of("mensagem", inserir(campos)));
            }
            case "PUT" -> {
                campos = Http.lerFormulario(ex);
                String msg = alterar(comId(id), campos);
                Http.enviarJson(ex, 200, Map.of("mensagem", msg));
            }
            case "DELETE" -> {
                String msg = excluir(comId(id));
                Http.enviarJson(ex, 200, Map.of("mensagem", msg));
            }
            default -> throw naoSuportado();
        }
    } catch (ApiException e) {
        Http.enviarJson(ex, e.status(), Map.of("erro", e.getMessage()));
    } catch (SQLException e) {
        int status = ErroSql.status(e);
        Http.enviarJson(ex, status, Map.of("erro", ErroSql.mensagem(e)));
    } catch (Exception e) {
        Http.enviarJson(ex, 500, Map.of("erro", "Erro interno: " + e.getMessage()));
    }
}
```

CrudHandler.java (resumido) — GET/POST/PUT/DELETE mapeiam para listar/inserir/alterar/excluir; três níveis de catch cobrem erro de validação, erro de SQL e qualquer outra falha.

Por que uma classe abstrata, e não um framework

Cada `*Handler` concreto (ex. `VooHandler`) sobrescreve só os métodos `listar/inserir/alterar/excluir` que fazem sentido para aquele recurso; o que não é sobrescrito lança 405 automaticamente (herdado de `CrudHandler`). Isso evita repetir o parsing de URL e o tratamento de erro em cinco handlers diferentes, sem precisar de um framework externo — é pouco código, mas resolve o problema real de forma direta.

Json e Http: utilitários sem dependências externas

`Json.java` é um serializador mínimo escrito à mão (cerca de 50 linhas) que converte `Map`, `Collection`, números, booleanos e texto em JSON, cuidando do escape de aspas e caracteres de controle. `Http.java` lê o corpo `application/x-www-form-urlencoded` enviado pelo front e escreve a resposta com o `Content-Type` correto. Como o projeto já usa JDK 17 sem libs externas na camada web, não haveria ganho em trazer uma biblioteca JSON só para isso.

ACESSO A DADOS

SQL explícito com PreparedStatement

Todo SQL do projeto roda através de `PreparedStatement`, nunca concatenando texto — isso é o que evita injeção de SQL. O ? marca onde um valor entra, e o valor é passado depois, como dado puro, não como parte do comando.

```
String sql = "DELETE FROM Passageiro WHERE cpf = ?";
try (Connection con = Database.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, cpf);
    if (ps.executeUpdate() == 0) {
        throw ApiException.naoEncontrado(
            "Passageiro com CPF " + cpf + " não encontrado.");
    }
} catch (SQLException e) {
    if (e.getErrorCode() == ErroSql.REFERENCIADO) {   // 1451
        throw new ApiException(409,
            "Este passageiro possui bilhetes comprados e não pode ser excluído.");
    }
    throw e;
}
```

PassageiroDAO.excluir() — o try-com-recursos fecha a conexão sozinho, mesmo se executeUpdate() lançar; o código 1451 vem direto do MySQL e é traduzido numa mensagem útil.

Transações: ou tudo, ou nada

Operações que mexem em mais de uma tabela (ou que dependem de uma ordem específica de comandos) desligam o auto-commit, executam os passos e só confirmam no fim. Se qualquer passo falhar, o `rollback()` desfaz tudo — o banco nunca fica num estado intermediário.

```
public void excluir(int numVoo) throws SQLException {
    try (Connection con = Database.getConnection()) {
        con.setAutoCommit(false);
        try {
            // 1) outros voos que apontam para este como conexão perdem a ligação
            String sqlSolta =
                "UPDATE Voo SET fk_Voo_conexao = NULL WHERE fk_Voo_conexao = ?";
            try (PreparedStatement ps = con.prepareStatement(sqlSolta)) {
                ps.setInt(1, numVoo);
                ps.executeUpdate();
            }
            // 2) remove o voo (a Escala é apagada em cascata; Bilhete bloqueia)
            try (PreparedStatement ps = con.prepareStatement(
                    "DELETE FROM Voo WHERE num_voo = ?")) {
                ps.setInt(1, numVoo);
                if (ps.executeUpdate() == 0) {
                    throw ApiException.naoEncontrado(
                        "Voo " + numVoo + " não encontrado.");
                }
            }
            con.commit();
        } catch (SQLException e) {
            con.rollback();
            if (e.getErrorCode() == ErroSql.REFERENCIADO) {
                throw new ApiException(409, "O voo " + numVoo
                    + " possui bilhetes vendidos e não pode ser excluído.");
            }
            throw e;
        }
    }
}
```

VooDAO.excluir() — o auto-relacionamento (Voo → Voo) exige dois passos: primeiro “soltar” quem referenciava este voo como conexão, depois apagar o voo em si.

O mesmo padrão aparece em `PassageiroDAO.inserir()` e `alterar()`: o passageiro e a lista de telefones (atributo multivalorado, tabela `Passageiro_Telefone`) são gravados na mesma transação — se a inserção dos telefones falhar, o passageiro recém-criado também é desfeito.

VALIDAÇÃO

Campos.java: a porteira antes do banco

Antes de qualquer dado chegar a um DAO, ele passa por `Campos`, uma classe pequena que lê o formulário recebido e levanta `ApiException` (erro 400) quando algo está errado — campo obrigatório ausente, texto longo demais, número inválido, data ou hora mal formatada. Isso mantém a validação de formato fora dos handlers e fora dos DAOs, num único lugar.

```
public int inteiro(String nome, String rotulo) {
    return converterInteiro(obrigatorio(nome, rotulo), rotulo);
}

public static int converterInteiro(String v, String rotulo) {
    try {
        return Integer.parseInt(v.trim());
    } catch (NumberFormatException e) {
        throw ApiException.invalido(
            "\"" + rotulo + "\" deve ser um número inteiro.");
    }
}

public String hora(String nome, String rotulo) {
    // HH:mm ou HH:mm:ss -> sempre devolve HH:mm:ss
    String v = opcional(nome);
    if (v == null) return null;
    try {
        LocalTime t = LocalTime.parse(v);
        return String.format("%02d:%02d:%02d",
            t.getHour(), t.getMinute(), t.getSecond());
    } catch (DateTimeParseException e) {
        throw ApiException.invalido("\"" + rotulo + "\" não é uma hora válida.");
    }
}
```

*Campos.java — cada *Handler monta seu objeto de domínio chamando esses métodos; regras de negócio mais específicas (ex. origem ≠ destino) ficam no próprio Handler.*

Validação de negócio no Handler

Regras que dependem de mais de um campo, ou que são específicas de um recurso, ficam no `Handler`, logo depois da leitura básica — por exemplo, `VooHandler.ler()` rejeita um voo cujo aeroporto de origem seja igual ao de destino, e rejeita um voo marcado como conexão dele mesmo. A constraint `CHECK` do banco (origem ≠ destino) também existe, como uma segunda linha de defesa — a aplicação valida antes por UX, o banco garante por trás.

DASHBOARD

Estatística calculada em Java sobre dados do banco

O dashboard combina SQL (para agregações simples) com cálculo em Java (para estatística descritiva). `DashboardDAO.montar()` roda várias consultas independentes — indicadores, voos por status, receita por companhia, ocupação por voo — e, à parte, lê todos os preços de bilhete para calcular média, mediana, desvio padrão e o histograma.

```
private Map<String, Object> estatisticas(List<Double> x) {
    int n = x.size();
    double soma = 0;
    for (double v : x) soma += v;
    double media = soma / n;

    double somaQuadrados = 0;
    for (double v : x) somaQuadrados += (v - media) * (v - media);
    double desvio = n > 1 ? Math.sqrt(somaQuadrados / (n - 1)) : 0;  // amostral

    double mediana = n % 2 == 1
            ? x.get(n / 2)
            : (x.get(n / 2 - 1) + x.get(n / 2)) / 2;
    ...
}
```

DashboardDAO.estatisticas() — desvio padrão amostral (divide por n–1, não por n); a lista já chega ordenada do SQL (ORDER BY preco), por isso a mediana só olha o meio.

```
int k = Math.max(3, Math.min(8, (int) Math.ceil(Math.sqrt(x.size()))));
double amplitude = (max - min) / k;
int[] contagem = new int[k];
for (double v : x) {
    int classe = (int) ((v - min) / amplitude);
    contagem[Math.min(classe, k - 1)]++;   // o valor máximo cai na última classe
}
```

DashboardDAO.histograma() — regra da raiz para o número de classes (entre 3 e 8), classes de mesma amplitude; o clamp evita estourar o array quando v == max.

Por que não deixar o MySQL calcular tudo

MySQL 8 tem funções estatísticas limitadas (não há STDDEV amostral nativo simples nem histograma pronto). Calcular em Java, sobre os preços já ordenados pelo próprio SQL, deixa claro no código qual é exatamente a fórmula usada — o que importa para a disciplina e facilita conferir o resultado à mão, como o grupo relata ter feito.

CONSULTAS

As 10 consultas SQL de ConsultaDAO

`ConsultaDAO` guarda, num `record Consulta(id, titulo, pergunta, conceitos, sql)`, as dez consultas que a aba “Consultas” da interface executa e mostra ao vivo — tanto o SQL quanto o resultado. Cada uma foi escolhida para demonstrar um recurso diferente de SQL:

#	Consulta	Conceito de SQL em destaque
1	Voos com todos os detalhes	JOIN de 6 tabelas; Aeroporto usado duas vezes (origem e destino)
2	Receita por companhia aérea	LEFT JOIN encadeado, GROUP BY, SUM/AVG, COALESCE
3	Passageiros acima da média	HAVING + subconsulta escalar sobre tabela derivada
4	Voos com aeronave em manutenção	EXISTS com subconsulta correlacionada
5	Taxa de ocupação por voo	LEFT JOIN, GROUP BY, aritmética com ROUND
6	Tripulação de cada voo	Tabela N:N (Escala), GROUP_CONCAT
7	Manutenções por mecânico	JOIN de especialização, DATEDIFF, CASE
8	Ranking de pilotos	Função de janela RANK() OVER
9	Bagagem acima da média	CTE (WITH) + subconsulta sobre a própria CTE
10	Voos com conexão e espera	Auto-JOIN (Voo com Voo), TIMEDIFF

```
SELECT RANK() OVER (ORDER BY p.horas_voo_acumuladas DESC) AS posicao,
       CONCAT(f.primeiro_nome, ' ', f.sobrenome) AS piloto,
       p.num_licenca, p.horas_voo_acumuladas,
       COUNT(v.num_voo) AS voos_comandados
FROM Piloto p
JOIN Funcionario f ON f.matricula = p.fk_Funcionario_matricula
LEFT JOIN Voo v    ON v.fk_Piloto_comandante = p.fk_Funcionario_matricula
GROUP BY p.fk_Funcionario_matricula, f.primeiro_nome, f.sobrenome,
         p.num_licenca, p.horas_voo_acumuladas
ORDER BY posicao
```

Consulta 8 (ranking de pilotos) — RANK() OVER é a única função de janela do projeto; o LEFT JOIN garante que um piloto sem voos ainda apareça no ranking.

No código, `executarTodas()` roda as dez consultas em sequência dentro de uma única conexão e **isola o erro de cada uma**: se uma consulta falhar, ela aparece na resposta com o campo `erro` preenchido, mas as outras nove continuam sendo executadas normalmente — só uma falha de conexão (`ErroSql.ehFalhaDeConexao`) interrompe tudo.

FRONT-END

HTML, CSS e JavaScript puro

A interface (`web/`) não usa framework de UI. `app.js` é organizado em pequenos módulos funcionais (um IIFE por aba: passageiros, voos, dashboard, consultas), todos construídos sobre um punhado de funções utilitárias reaproveitadas pelas quatro abas.

```
async function api(metodo, url, dados) {
  const opcoes = { method: metodo };
  if (dados) opcoes.body = new URLSearchParams(dados);
  const resp = await fetch(url, opcoes);
  const corpo = await resp.json().catch(() => ({}));
  if (!resp.ok) throw new Error(corpo.erro || `Erro ${resp.status}`);
  return corpo;
}

function el(tag, props = {}, ...filhos) {
  const e = document.createElement(tag);
  for (const [k, v] of Object.entries(props)) {
    if (k.startsWith('on')) e.addEventListener(k.slice(2), v);
    else if (k === 'class') e.className = v;
    else e.setAttribute(k, v);
  }
  for (const f of filhos) {
    e.append(f instanceof Node ? f : document.createTextNode(String(f)));
  }
  return e;
}
```

app.js — api() centraliza toda chamada fetch e já lança um erro com a mensagem vinda do backend; el() monta elementos usando textContent internamente, evitando XSS por template string.

Um formulário, dois modos

`controlarFormulario()` é o padrão que os formulários de Passageiro e Voo compartilham: o mesmo `<form>` alterna entre “novo” e “editando”, travando o campo de chave primária durante a edição para não deixar o usuário mudar um CPF ou número de voo já salvo pelo formulário de update.

Dashboard e Chart.js

O dashboard busca um único JSON em `GET /api/dashboard` (exatamente a estrutura montada por `DashboardDAO.montar()`) e alimenta quatro gráficos Chart.js mais os cartões de estatística, sem nenhuma chamada adicional — todo o trabalho pesado (agregação SQL e estatística) já aconteceu no servidor.

FECHAMENTO

Como rodar e o que vem a seguir

Executando o projeto

- Pré-requisitos: **JDK 17+** e **MySQL 8+**.
- Rodar, nesta ordem, `database/migrations.sql` (cria o banco e as tabelas) e `database/queries.sql` (insere os dados).
- Copiar `config.properties.example` para `config.properties` com usuário/senha do MySQL local.
- Compilar e rodar com `./run.sh` (Linux/macOS/Git Bash) ou os comandos equivalentes de `javac`/`java` no PowerShell (documentados no README).
- Abrir `http://localhost:8080`.

Pontos fortes do código, em resumo

- SQL sempre parametrizado — nenhuma concatenação de string vira comando SQL.
- Erros do MySQL viram mensagens legíveis em vez de stack trace (ErroSql + ApiException).
- Operações multi-tabela são transacionais; nada fica “meio salvo”.
- Uma única classe (CrudHandler) resolve o roteamento REST para cinco recursos diferentes.
- Zero dependências externas na camada web (sem framework HTTP, sem lib JSON) — só JDK e o driver JDBC do MySQL.

Próximos passos (conforme o README do projeto)

- Consultas avançadas, visões e índices adicionais.
- Funções, procedimentos armazenados e triggers — por exemplo, garantir por trigger que o assento de um bilhete pertence à aeronave do voo.
- Evoluções na interface final e no dashboard.

Sobre este documento

Elaborado a partir do código-fonte atual do repositório (pacotes `aeroporto.db`, `aeroporto.model`, `aeroporto.dao` e `aeroporto.web`, mais `database/migrations.sql` e `web/`), com foco na explicação técnica da implementação para apoiar o entendimento e a manutenção do projeto.